

BCSE204L Design and Analysis of Algorithms

Module-2

Topic-5:Backtracking

Backtracking Basics

- Another algorithm design paradigm
- Uses brute-force
- Try all solutions and pick the desired ones
- DP also does the same??
- But DP is for optimization problems
- Backtracking is NOT for optimization problems

Backtracking

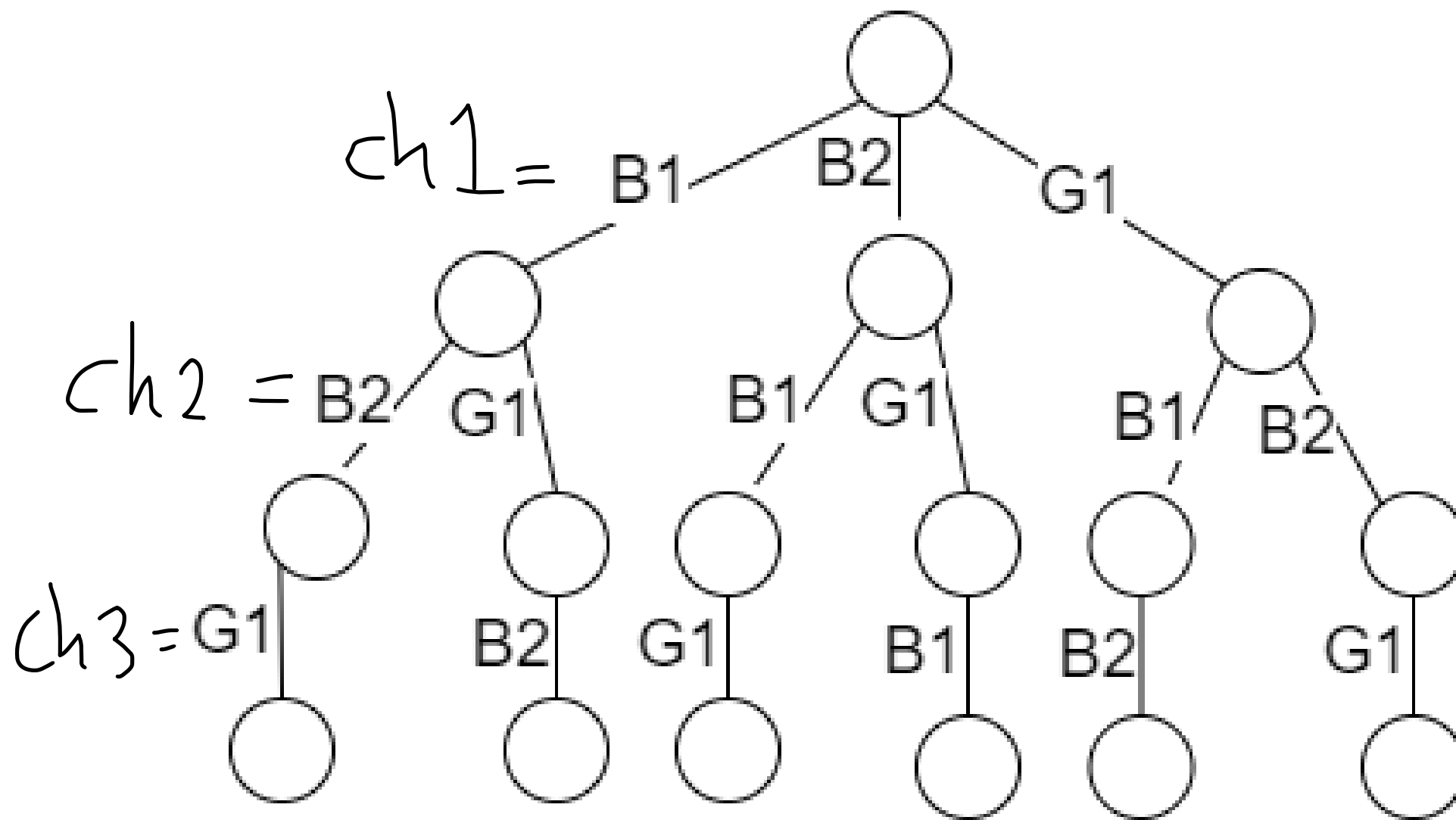
- Is used when a problem has multiple solutions, and we want all of them
- The solutions are restricted by either implicit or explicit constraints
 - ✓ **Explicit Constraint:** Explicit constraints are rules that restrict each x_i to take on values only from a given set. Ex: $x_n = 0$ or 1 , 4 queens can be placed in 4x4 chess board in 4^4 ways(256)
 - ✓ **Implicit Constraint:** Implicit Constraints are rules that determine which of the tuples in the solutions space of will satisfy the criterion function
- Backtracking can be explained as a modified depth first search tree
- Backtracking is a procedure whereby, after determining that a node can lead to nothing but dead end, we go back (backtrack) to the parent node and proceed with the search on the next child

An Example: No implicit constraints

- We have 3 children:
- 1 Girl (G1) and 2 Boys (B1 and B2)
- We have 3 chairs
- In how many ways the children can sit in the chairs
- There are $3! = 6$ possible solutions
- We want all, can be represented as a tree – *state space tree*

Chair 1	Chair 2	Chair 3
---------	---------	---------

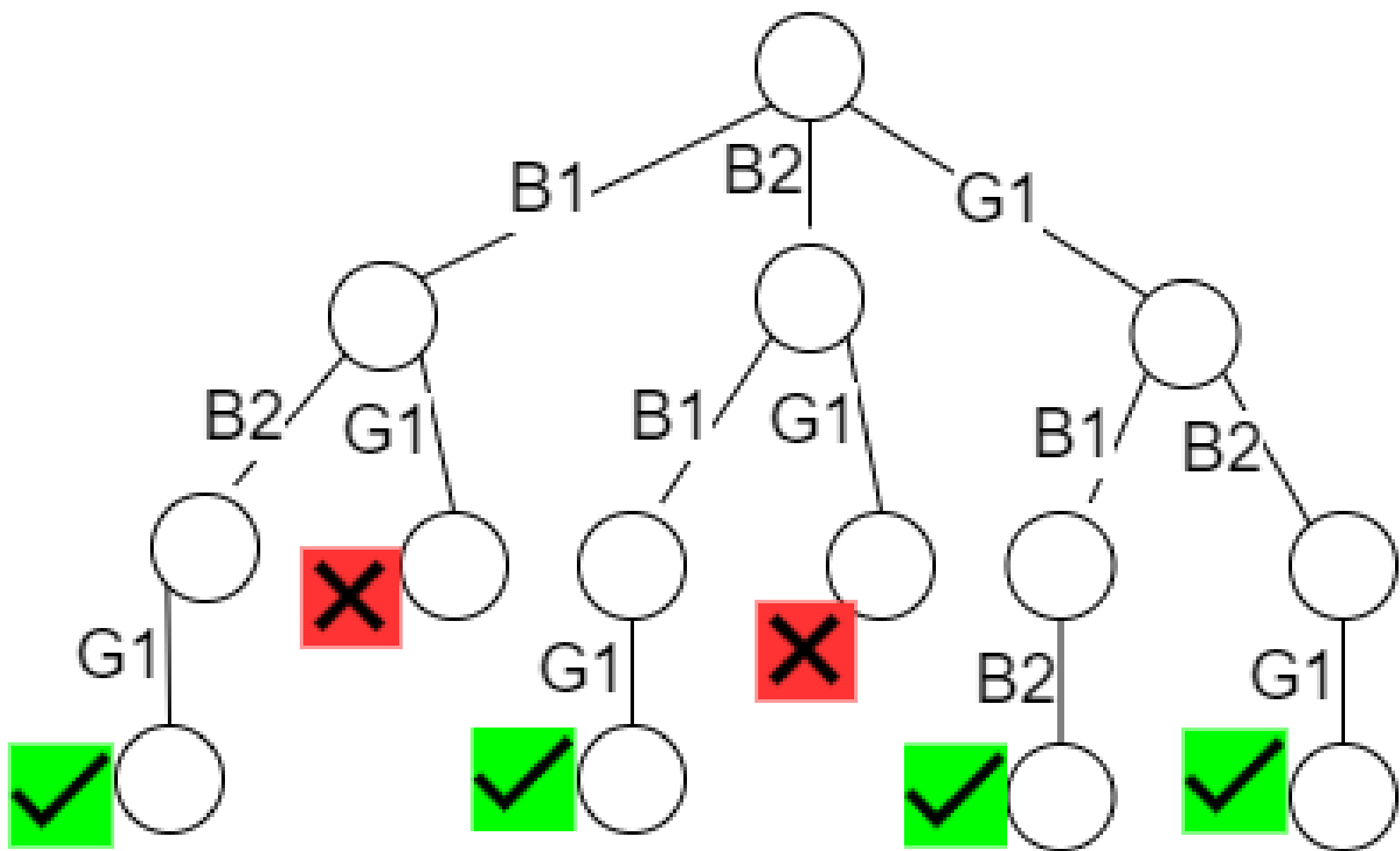
G1 B1 B2



- We use a **state space tree** to represent the solutions
- Many a times the problem would be to find only those solutions which satisfy some implicit constraints
- In that case the we kill the nodes by applying the **bounding function** and then backtrack

Example: with implicit constraints

- Impose the constraint that the girl cannot sit in the middle
- We develop the state space tree as in the previous case, but we will apply the bounding function and kill the nodes which do not proceed to a solution
- Total 4 solutions, we want all



Terminology:

Problem state is each node in the depth-first search tree

State space is the set of all paths from root node to other nodes

Solution states are the problem states s for which the path from the root node to s

Answer states are that solution states s for which the path from root node to s defines a tuple that is a member of the set of solutions

State space tree is the tree organization of the solution space

Live node is a generated node for which all of the children have not been generated yet.

E-node is a live node whose children are currently being generated or explored

Dead node is a generated node that is not to be expanded any further

n-Queens Problem(4-Queens and 8-Queens Problem)

- Consider a $n \times n$ chess board.
- Let there are n Queens. These n Queens are to be placed on $n \times n$ chess board so that no two queens are on the same column, same row or same diagonal, ie. no two queens can attack
- Consider 4x4 chess board, there are $16C_4$ possible ways to place 4 Queens using brute force approach.

Place first queen in the first row
in the first column.

As it is the first queen it is
not under attack.

$X[1]=1$ (column value is assigned)

	1	2	3	4
1	1			
2				
3				
4				

$X[1]=1$

To place second queen in second row
Start with first column.

It is under attack

Second column also Under attack

Third column not under attack by other queens.

So we place queen in 3rd column.

$X[2]=3$

	1	2	3	4
1	1			
2	-	-	2	
3				
4				

$X[2]=3$

To place third queen in third row
first col under attack
second column under attack
third column under attack
fourth column under attack
not possible to place queen in third row
because placement of previous queens is
not correct. So backtrack to previous row
and move the queen to another possible place
and continue.

	1	2	3	4
1	1			
2			2	
3	-	-	-	-
4				

Go to second row
Move the queen to another col.
Another possibility is column 4.
Move to col 4.
Now $X[2]=4$

	1	2	3	4
1	1			
2	-	-	-	2
3				
4				

$X[2]=4$

Go to third row to place 3rd queen

First col under attack

Second column not under attack by other queens

So place the queen in 2nd col.

X[3]=2

	1	2	3	4
1	1			
2				2
3	-	3		
4				

X[3]=2

Now to place 4th queen in 4th row

First col under attack by other queen

Second col under attack by other queen

Third col under attack by other queen

Fourth col under attack by other queen

Not possible to place the queen in 4th row as there is a problem in the placement of previous queens

Back track to previous placements

Goto 3rd row and try to move the queen to another place.

The other places are under attack go to 2nd row

Already we checked all possibilities in 2nd row

we backtrack to first row.

	1	2	3	4
1	1			
2				2
3		3		
4	-	-	-	-

First queen is moved to 2nd column
X[1]=2

	1	2	3	4
1		1		
2				
3				
4				

X[1]=2

Second queen in second row
First col under attack
Second column under attack
Third col under attack
4th col not under attack
So place queen in 4th col
X[2]=4

	1	2	3	4
1		1		
2	-	-	-	2
3				
4				

X[2]=4

To place third queen in third row
First col not under attack
So place the queen in first col
X[3]=1

	1	2	3	4
1		1		
2				2
3	3			
4				

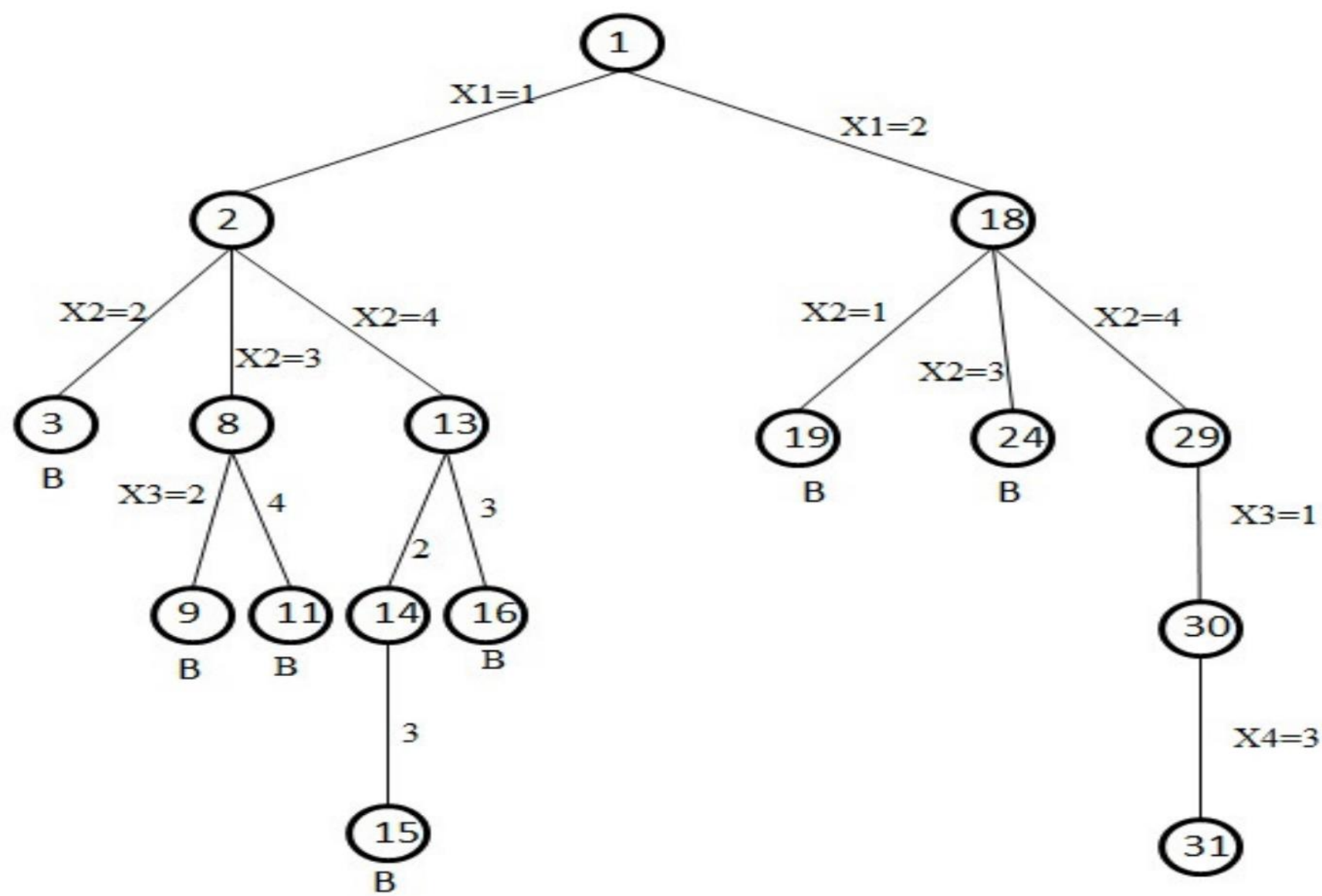
X[3]=1

To place 4th queen in 4th row
first col under attack
second col under attack
third col not under attack
so place the 4th queen in 3rd col
x[4]=3

	1	2	3	4
1		1		
2				2
3	3			
4	-	-	4	

X[4]=3

All four queens are placed in the
4x4 chess board without attacking each other.
In the same way it is possible to place all 8 queens in
an 8x8 chess board without attacking each other.



Tracing of the algorithm to place 4 queens on a 4x4 cross board such that no two queens attack each other.

```
nqueens(k,n)      place(k,i)
```

```
nqueens(1,4)    place(1,1) returns True so x[1]=1
```

```
nqueens(2, 4)    place(2, 1) returns False
                  place(2, 2) returns False
                  place(2, 3) returns True so X[2]=3
```

```
nqueens(3, 4)    place(3, 1) returns False
                  place(3, 1) returns False
                  place(3, 1) returns False
                  place(3, 1) returns False
                                     Backtracking
```

nqueens(2,4) place(2,4) returns True so X[2]=4

```
nqueens(3, 4)    place(3, 1) returns False
                  place(3, 2) returns True so X[3]=2
```

```
nqueens(4, 4)    place(4, 1) returns False
                  place(4, 2) returns False
                  place(4, 3) returns False
                  place(4, 4) returns False
                                Backtracking
```

nqueens(1, 4) place(1, 2) returns True so x[1]=2

nqueens(2, 4) place(2, 1) returns False
 place(2, 2) returns False
 place(2, 3) returns False
 place(2, 4) returns True so X[2]=4

nqueens(3, 4) place(3, 1) returns True so X[3]=1

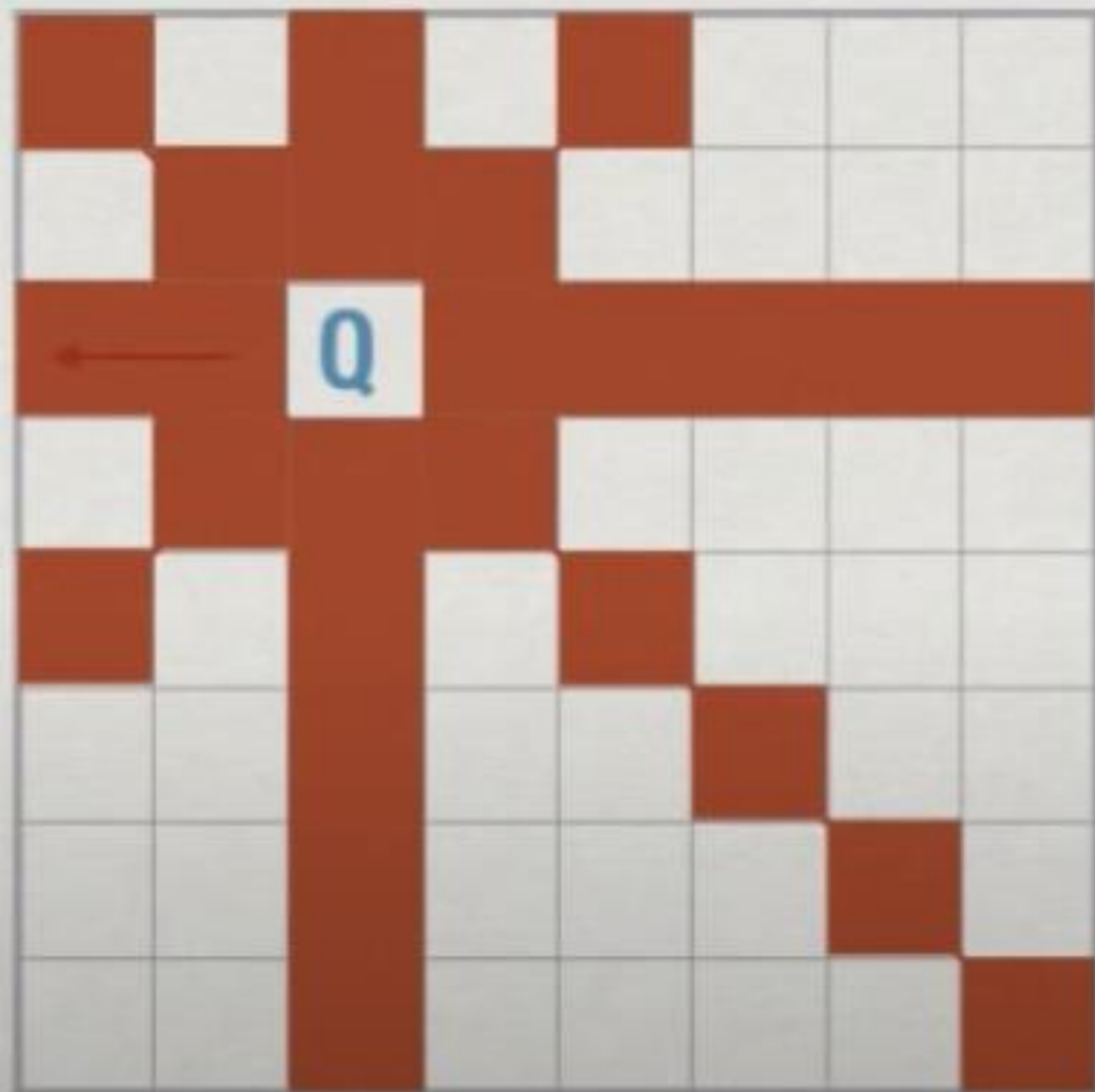
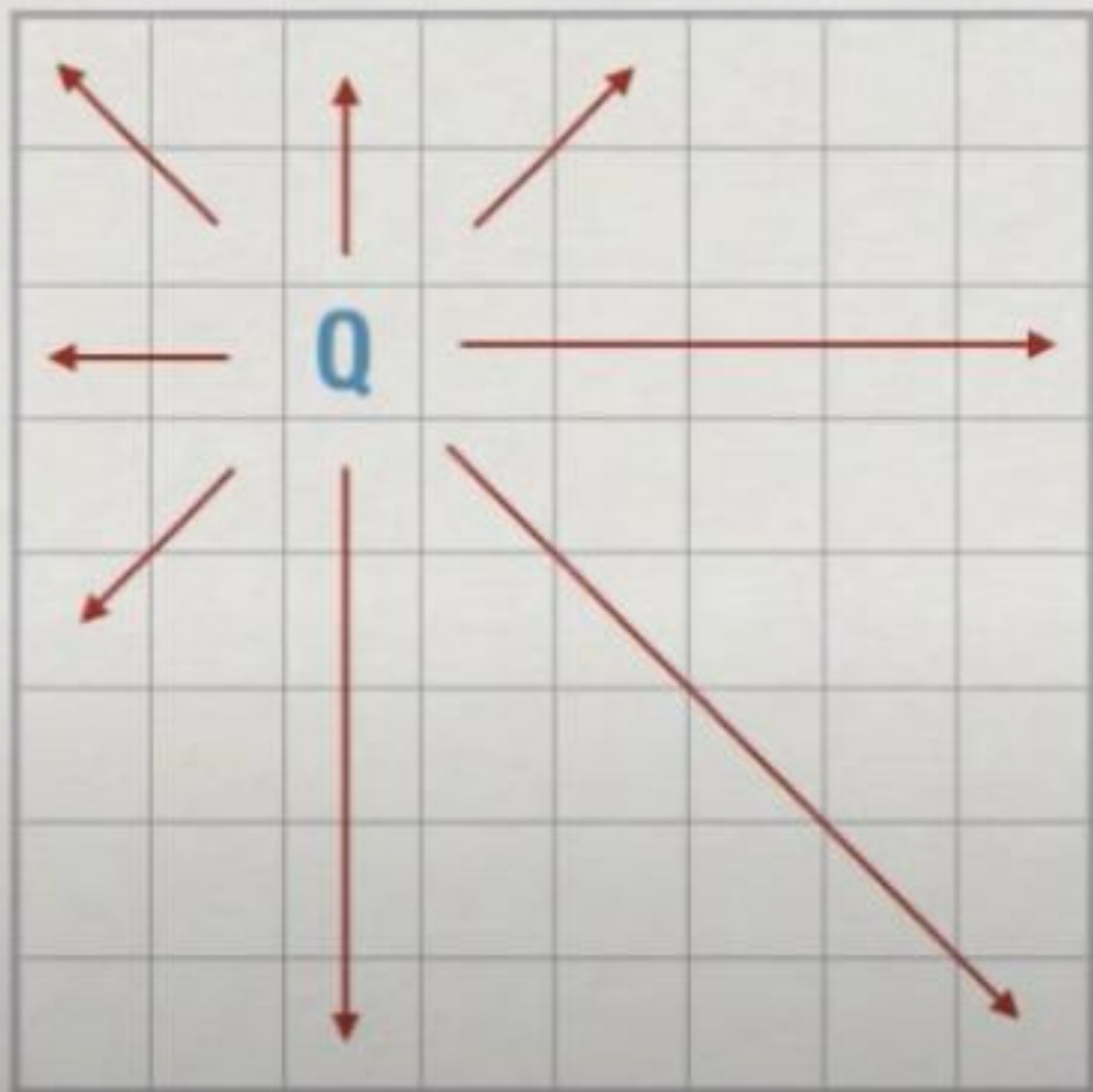
nqueens(4, 4) place(4, 1) returns False
 place(2, 2) returns False
 place(2, 3) returns True so X[4]=3

The solution vector for a 4x4 cross board to place 4 non attacking queens is

x[1]=2
x[2]=4
x[3]=1
x[4]=3

8 Queens Problem

- Place 8 queens on a chess board so that none of them attack each other
- In chess, a queen can move any number of squares along a row, column or diagonal
- 8 is clearly the limit



Checking diagonal attacks

- If two queens are placed at position (i, j) and (k, l)
- Then they are on same main diagonal (NW to SE) only if $(i - j) = (k - l)$ or $(j - l) = (i - k)$ and
- They are on same off diagonal (SW to NE) only if $(i + j) = (k + l)$ or $(j - l) = (k - i)$
- Combining both
- $|j - l| = |i - k|$

```

Algorithm NQueens( $k, n$ )
// Using backtracking, this procedure prints all
// possible placements of  $n$  queens on an  $n \times n$ 
// chessboard so that they are nonattacking.
{
    for  $i := 1$  to  $n$  do
    {
        if Place( $k, i$ ) then
        {
             $x[k] := i$ ;
            if ( $k = n$ ) then write ( $x[1 : n]$ );
            else NQueens( $k + 1, n$ );
        }
    }
}

```

```

Algorithm Place( $k, i$ )
// Returns true if a queen can be placed in  $k$ th row and
//  $i$ th column. Otherwise it returns false.  $x[ ]$  is a
// global array whose first  $(k - 1)$  values have been set.
// Abs( $r$ ) returns the absolute value of  $r$ .
{
    for  $j := 1$  to  $k - 1$  do
        if (( $x[j] = i$ ) // Two in the same column
            or ( $\text{Abs}(x[j] - i) = \text{Abs}(j - k)$ ))
            // or in the same diagonal
        then return false;
    return true;
}

```

Time Complexity

- $T(n) = nT(n - 1) + O(n^2)$
- Because n recursive calls of $T(n - 1)$ and *Place* function, which is of $O(n)$ complexity would be called n times
- Solving the recurrence,
 - Time complexity: The solution space tree of 8-queens problem contains 8^8 tuples. After imposing implicit constraints, the size of solution space is reduced to $8!$ tuples.
- $O(n!) = O(n^n)$

References

- NPTEL Course on “Programming, Data Structures and Algorithms Using Python” by Prof. Madhavan Mukund, of Chennai Mathematical Institute (CMI)

Thank you..!!